Fog and Cloud Computing

Better Buses

Team members
Andrea Jesus Soranzo Mendez (267339)
Andrea Precoma (267071)

University of Trento - A.Y. 2025/2026

Table of content

1. Abstract

Urban mobility faces ongoing challenges with recurring congestion and inefficient public transit scheduling. To address this problem, this project proposes a comprehensive network of IoT sensors deployed across the urban bus fleet in Trento. These sensors, embedded within every public bus and at various bus stops, continuously capture real-time traffic and transit data on an hourly basis. The system calculates delays and gathers critical metrics to identify peak traffic periods and congestion hotspots throughout the city. This can help to optimize transit routes and updates schedules to help buses avoid traffic and stay on time. It also guides strategic infrastructure planning, allowing engineers to build targeted solutions in high-traffic zones. Finally, it enhances the passenger experience by powering mobile apps with real-time tracking and accurate arrival predictions.

2. Architecture

To realize this project, we selected a three-tier Fog Computing architecture. This hybrid approach balances local processing with heavy cloud analytics to ensure low latency, efficient bandwidth usage, and high scalability. The main task of the sensors is just to collect all the relevant data and then send them to a local and centralized server that does all the computing and calculations. After that, the data is sent to the cloud, which comprehends all the data and provides a monitoring interface with graphs in order to make all information humanly readable.

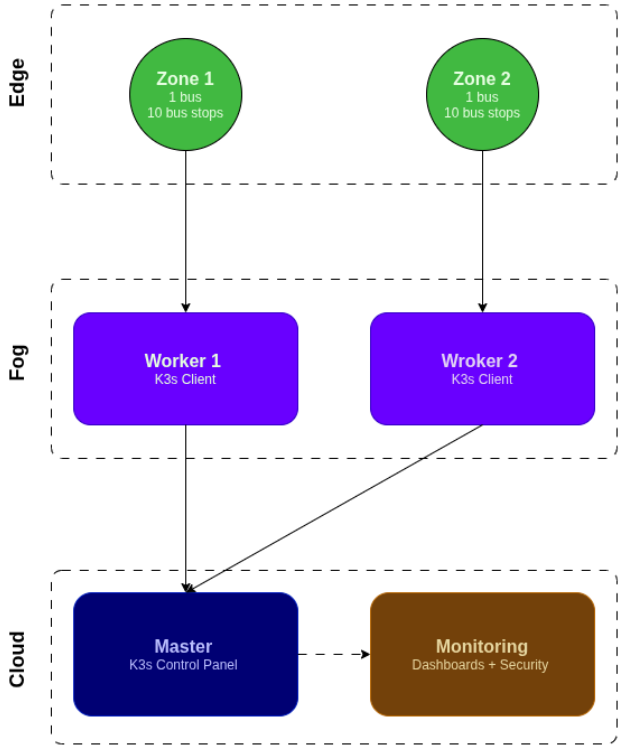

Figure 1: System architecture

2.1. Layers

2.1.1. Edge Layer

The primary role of the bus-mounted sensors is localized data ingestion. They act as lightweight edge devices responsible for continuously collecting raw transit data (such as GPS coordinates, time stamps, and speed

variations) without overloading local processing capacity. Instead those mounted in the stops serves like a checkpoint station in order to track automatically at which stop and what time a bus arrived.

2.1.2. Fog Layer

Once collected, these raw data packages are transmitted to localized fog nodes and regional servers. This layer performs the initial data filtering, aggregation, and critical real-time calculations, such as computing immediate route delays. By handling processing at the fog level, we minimize data transmission costs and enable rapid localized response times.

2.1.3. Cloud Layer

Finally, the pre-processed data is forwarded to a centralized cloud platform. The cloud orchestrates large-scale data aggregation, historical analysis, and can run predictive machine learning models. Crucially, the cloud layer hosts a comprehensive monitoring interface, converting complex datasets into human-readable dashboards, real-time graphs, and predictive analytics for stakeholders and urban planners.

2.2. Technology Stack

Speaking of the implementation we decided to mainly stick with those discovered and studied during the lectures.

2.2.1. OpenNebula

OpenNebula serves as the core cloud hypervisor and cloud management platform, orchestrating the creation, deployment, and management of our virtualized infrastructure. Given the scale of the physical deployment, OpenNebula allows us to build a high-fidelity simulation environment by provisioning distinct Virtual Machines to emulate each physical component of our architecture. This virtualized ecosystem is divided into three node configurations, created in the following order:

- **Master VM** - the machine that hosts K3s in server mode and the primary services, with IP `172.16.100.2`.
- **Sensors VM** - a lightweight machine used only for simulation purposes, generating fake data from sensors divided into two areas. The assigned IP is `172.16.100.3`.
- **Worker-Z** - the machine that hosts the K3s in client mode, namely the worker in charge of elaborating data from the sensors in the zone Z. In our project we define two areas, hence two workers. However, since the amount of workers may vary, their IP is, generally speaking, `172.16.100.x` where $x = 3 + Z$.

2.2.2. Kubernetes

To manage our data-processing applications within the fog layer, we deploy a Kubernetes cluster directly on top of the virtualized infrastructure provisioned by OpenNebula. This container orchestration strategy separates the infrastructure into a centralized Control Plane and a resilient Worker Layer. A single, dedicated OpenNebula virtual machine is allocated to act exclusively as the Kubernetes Control Plane (Master Node). This node serves as the brain of the cluster, executing core components such as scheduler and controller manager. The other VMs are configured as Kubernetes Worker Nodes. These nodes will host the application workloads encapsulated within pods. These worker pods execute the microservices responsible for real-time data processing and filtering. By leveraging Kubernetes on top of our fog nodes, the system inherits robust self-healing mechanisms to ensure uninterrupted data streams and guarantees pod replication and workload rescheduling.

2.2.3. Prometheus and Grafana

To bridge the gap between complex data streams and actionable insights, we deploy the Prometheus and Grafana monitoring stack within our centralized cloud layer. Prometheus serves as the core time-series database and

monitoring toolkit, actively scraping and storing metrics related to both infrastructure health (e.g. node status, resource consumption) and application-level sensor data. Grafana acts as our universal data visualization hub, directly querying Prometheus to aggregate these disparate metrics and present them through an intuitive, real-time, and human-readable web interface.

2.2.4. Security

For the security aspect we decided to adopt two main level of security.

2.2.4.1. Container Hardening

To minimize the attack surface of our data-processing microservices, we apply strict container hardening techniques. This ensures that each container is as resilient as possible against exploitation:

- **Privilege escalation prevention** - containers are configured to run as non-root users, stripping away unnecessary administrative capabilities.
- **Minimal image footprint** - we utilize lightweight base images to remove unnecessary binaries, libraries, and functionalities that could be leveraged by an attacker.

2.2.4.2. Kubernetes Network Policies

By implementing Network Policies, we enforce a "Zero Trust"-like posture between services:

- **Namespace isolation** - distinct environments are isolated into dedicated namespaces.
- **Granular traffic control** - we define explicit "allow-lists" for Pod-to-Pod communication (Ingress/Egress lists). This ensures that a compromised sensor-data pod cannot laterally communicate with the visualization database or the control plane unless explicitly authorized.

2.2.4.3. OpenNebula Security Groups and Virtual Firewalls

Another layer of defense is managed at the hypervisor level through OpenNebula security groups, specifically designed per each type of VM. **Master-SG** and **Worker-SG** are created and assigned to the Master VM and Worker VMs respectivelly, each one allowing inbound traffic from specific IP addresses through specific ports according to their role in the cluster. The default security group is left to the Sensors VM since it is used for simulation only.

2.2.4.4. Falco and Falcosideckick

Falco operates as an advanced behavioral activity monitor designed to detect anomalous activity. In out case it continuously observes for:

- **Remote shell opening** - detecting if a terminal is unexpectedly spawned inside a running pod, specifically the one in charge of running MQTT.
- **Accessing sensitive host files** - monitoring if a container attempts to read host-level files outside its authorized scope (e.g. /etc/passwd), suggesting an attacker is trying to perform privilege escalation or gather critical system information.
- **Establishing unexpected outbound connections** - ensuring that pods strictly adhere to their intended communication paths.

To enhance incident response and alert management, Falcosidekick is deployed alongside Falco to seamlessly process, aggregate, and forward these real-time security alerts to our centralized monitoring channels.

2.2.4.5. Reverse Proxy

Nginx is deployed as a reverse proxy to secure external access to the Prometheus monitoring interface, as Prometheus lacks built-in authentication mechanisms by default.

2.3. Labels and namespaces

To optimize sensor management and enforce strict scheduling rules within our Kubernetes cluster, we implement node selector policies by assigning targeted labels to our nodes. We defined two primary labels based on the workloads' operational responsibilities:

- **Workers** - label assigned to nodes handling traffic data analysis. These workloads track where buses experience delays or prolonged stops, correlating the data with specific times and days to calculate traffic averages.
- **Zone-Z** - namespace assigned to nodes responsible for monitoring data that comes from a specific zone Z in the city (e.g zone-1). This includes both types of sensors.
- **Falco** - namespace assigned to pods that run an instance of falco and falcosideckick in order to check e trigger alerts.

3. Implementation

To validate our architectural design, we constructed a fully functional simulation environment focused on dynamic scaling, localized edge processing, and operational resilience.

3.1. Infrastructure Setup

To simulate a distributed fog network on a local machine, we utilized a nested virtualization strategy. A primary Ubuntu Server 24.04 VM (via VirtualBox) acts as the central controller, with hardware nested virtualization enabled to allow OpenNebula to utilize KVM internally.
To ensure reproducibility, the deployment avoids manual configuration in favor of a highly automated, modular Git-based architecture. The repository is divided into four isolated branches, each bootstrapping a specific component via OpenNebula's contextualization scripts (`start-script.sh`):

- **Host** - manages the foundational IaaS layer. It automates the provisioning of OS images, VM templates, OpenNebula Security Groups, and the VMs themselves. It also configures port forwarding (30000 for Grafana, 30001 for Prometheus) to expose the centralized monitoring dashboards outside the virtualized network.
- **Master-node** (Cloud Layer) - establishes the Kubernetes control plane by installing K3s in server mode. It automatically deploys the core infrastructure stack: Prometheus, Grafana, Nginx, and Falco. Once the cluster is active, a dedicated orchestration script deploys Mosquitto brokers, Telegraf pods, and network policies, utilizing Kubernetes node labels to assign these workloads specifically to the edge nodes.
- **Worker-node** (Fog Layer) - dynamically provisioned by OpenNebula as edge processors, these VMs automatically configure K3s in client mode to seamlessly join the Kubernetes cluster.
- **Sensors** - dedicated VM running Python simulators to replicate urban telemetry. It models two distinct areas (each with one bus and five stops), injecting stochastic delays upon arrival at bus stops to accurately mimic real-world traffic fluctuations and passenger boarding friction.